Assignment14.4

Task 1: Responsive Homepage Layout for a

Startup Website

Scenario

You are designing a basic homepage for a startup company that needs

to look good on desktops, tablets, and mobile devices.

index.html > html

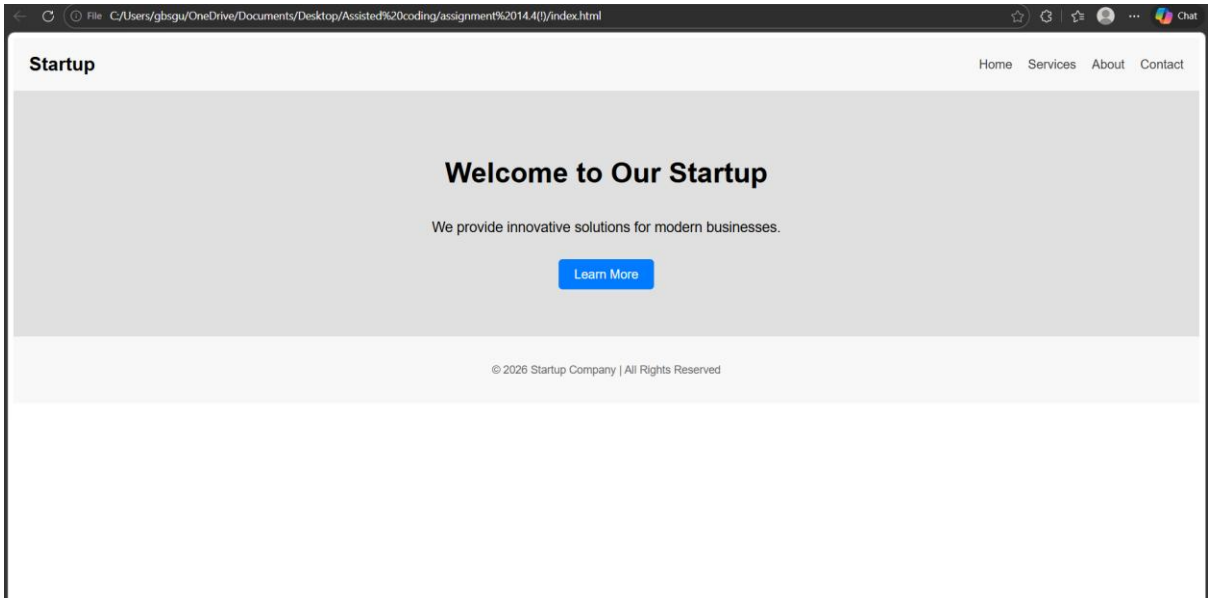
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Startup Website</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

<header>
    <div class="logo">Startup</div>
    <nav>
        <a href="#">Home</a>
        <a href="#">Services</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </nav>
</header>

<main>
    <section class="hero">
        <h1>Welcome to Our Startup</h1>
        <p>We provide innovative solutions for modern businesses.</p>
        <button>Learn More</button>
    </section>
</main>

<footer>
    <p>© 2026 Startup Company | All Rights Reserved</p>
</footer>

</body>
</html>
```

style.css > header

```css
#Generate responsive CSS for a startup homepage with a header containing a logo and navigation links, a main hero section with title, description and button, and a footer
/* Base styles */
*{
    margin:0;
    padding:0;
    box-sizing:border-box;
}

body{
    font-family: Arial, sans-serif;
}

header{
    display:flex;
    justify-content:space-between;
    align-items:center;
    padding:20px;
    background-color:#f8f8f8;
}

.logo{
    font-size:24px;
    font-weight:bold;
}

nav{
    display:flex;
    gap:20px;
}

nav a{
    text-decoration:none;
    color:#333;
    font-size:16px;
}

.hero{
```

```
nav a{
    font-size:16px;
}

.hero{
    display:flex;
    flex-direction:column;
    align-items:center;
    text-align:center;
    padding:60px 20px;
    background-color:#e0e0e0;
}

.hero h1{
    font-size:36px;
    margin-bottom:20px;
}

.hero p{
    font-size:18px;
    margin-bottom:30px;
}

.hero button{
    padding:10px 20px;
    font-size:16px;
    color:#fff;
    background-color:#007BFF;
    border:none;
    border-radius:5px;
    cursor:pointer;
}

footer{
    text-align:center;
    padding:20px;
    background-color:#f8f8f8;
    font-size:14px;
    color:#666;
}

/* Tablet */
@media (max-width:768px){

    header{
        flex-direction:column;
        gap:10px;
    }

    nav{
        flex-direction:column;
        align-items:center;
        gap:10px;
    }
```

```
style.css > header
.hero button{
    background-color:#007BFF;
    border:none;
    border-radius:5px;
    cursor:pointer;
}

footer{
    text-align:center;
    padding:20px;
    background-color:#f8f8f8;
    font-size:14px;
    color:#666;
}

/* Tablet */
@media (max-width:768px){

    header{
        flex-direction:column;
        gap:10px;
    }

    nav{
        flex-direction:column;
        align-items:center;
        gap:10px;
    }

    .hero h1{
        font-size:30px;
    }
}

/* Mobile */
@media (max-width:480px){

    .hero{
        padding:40px 15px;
    }

    .hero h1{
        font-size:24px;
    }

    .hero p{
        font-size:16px;
    }

}
```



Task 2: Interactive Call-to-Action Button

Scenario

A marketing page requires a call-to-action button that responds when

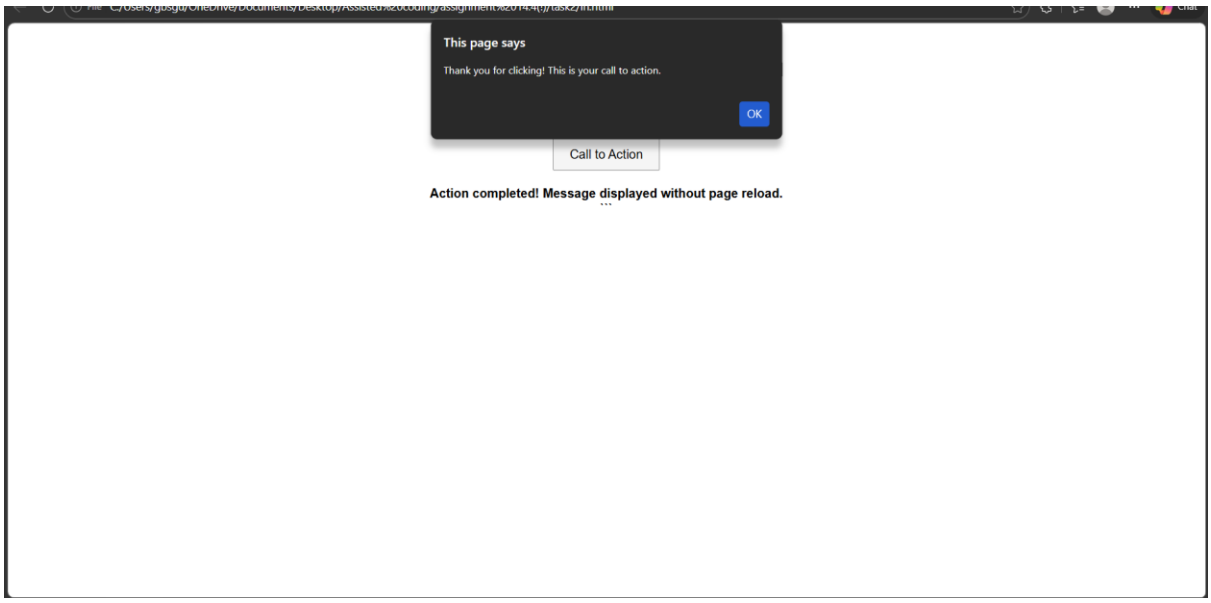
users interact with it.

Generate JavaScript for a call-to-action button that shows an alert and displays a message on the page when clicked without reloading the page and include meaningful comments explaining the logic.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

in.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Call-to-Action Button</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
        }
        button {
            padding: 10px 20px;
            font-size: 16px;
            cursor: pointer;
        }
        #message {
            margin-top: 20px;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>Click the Button!</h1>
    <button id="ctaButton">Call to Action</button>
    <div id="message"></div>

    <script>
        // Get references to the button and message display elements
        const ctaButton = document.getElementById('ctaButton');
        const messageDiv = document.getElementById('message');

        // Add an event listener to the button for the 'click' event
        ctaButton.addEventListener('click', function() {
            // Show an alert popup with a message
            alert('Thank you for clicking! This is your call to action.');

            // Display a message on the page without reloading
            // Update the innerHTML of the message div to show the text
            messageDiv.innerHTML = 'Action completed! Message displayed without page reload.';
        });
    </script>
</body>
</html>
```



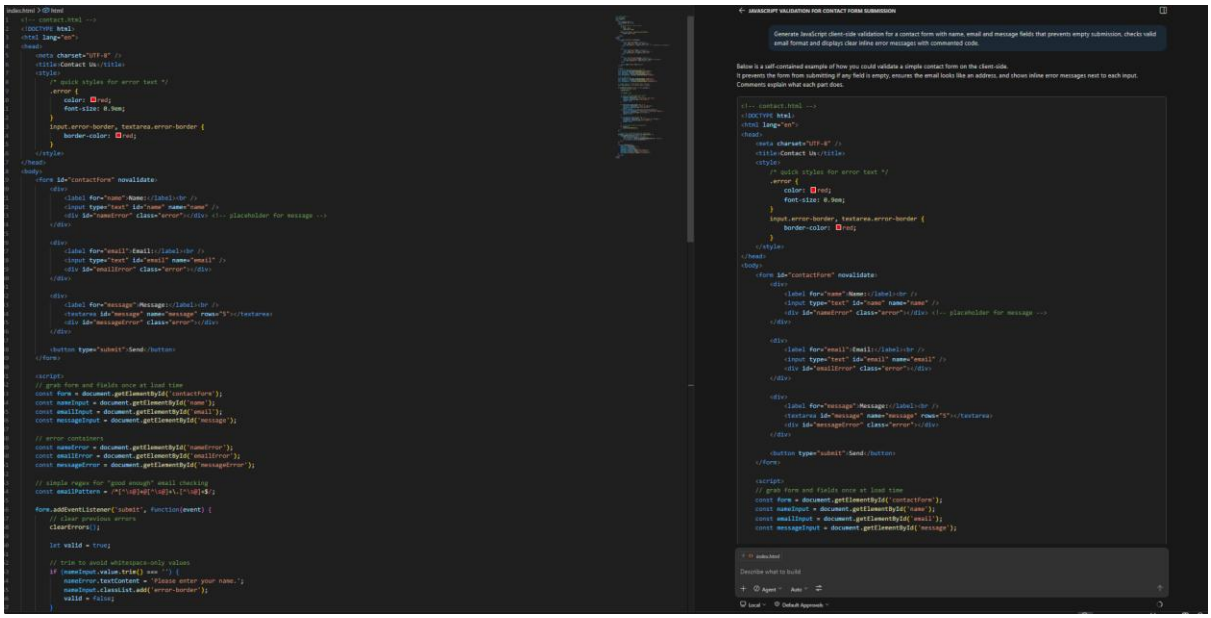
Task 3: Contact Form with Client-Side

Validation

Scenario

You are developing a contact form for a company website to collect

user queries safely and correctly.



```html
<html lang="en">
<body>
    <script>
    const emailInput = document.getElementById('email');
    const messageInput = document.getElementById('message');

    // error containers
    const nameError = document.getElementById('nameError');
    const emailError = document.getElementById('emailError');
    const messageError = document.getElementById('messageError');

    // simple regex for "good enough" email checking
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    form.addEventListener('submit', function(event) {
        // clear previous errors
        clearErrors();

        let valid = true;

        // trim to avoid whitespace-only values
        if (nameInput.value.trim() === '') {
            nameError.textContent = 'Please enter your name.';
            nameInput.classList.add('error-border');
            valid = false;
        }

        if (emailInput.value.trim() === '') {
            emailError.textContent = 'Email is required.';
            emailInput.classList.add('error-border');
            valid = false;
        } else if (!emailPattern.test(emailInput.value.trim())) {
            emailError.textContent = 'Please enter a valid email address.';
            emailInput.classList.add('error-border');
            valid = false;
        }

        if (messageInput.value.trim() === '') {
            messageError.textContent = 'Don\'t forget your message.';
            messageInput.classList.add('error-border');
            valid = false;
        }

        // if any check failed, stop submission
        if (!valid) {
            event.preventDefault();
        }
    });

    // remove error text/borders when user types again
    [nameInput, emailInput, messageInput].forEach(elem => {
        elem.addEventListener('input', () => {
            elem.classList.remove('error-border');
            const errorDiv = document.getElementById(elem.id + 'Error');
            if (errorDiv) errorDiv.textContent = '';
        });
    });

    function clearErrors() {
        nameError.textContent = '';
        emailError.textContent = '';
        messageError.textContent = '';
        nameInput.classList.remove('error-border');
        emailInput.classList.remove('error-border');
        messageInput.classList.remove('error-border');
    }
    </script>
```

Name:
Eaga Rushikesh
Email:
2303a51634@sru.edu.in
Message:
hi
Send

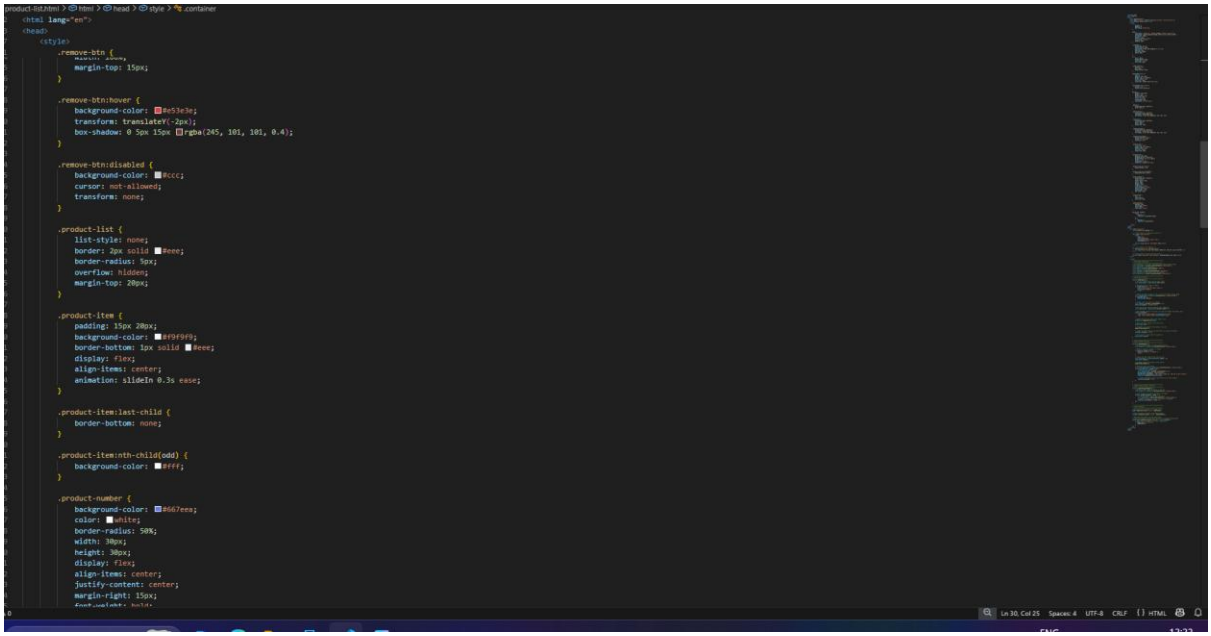
Task 4: Dynamic List Management for a

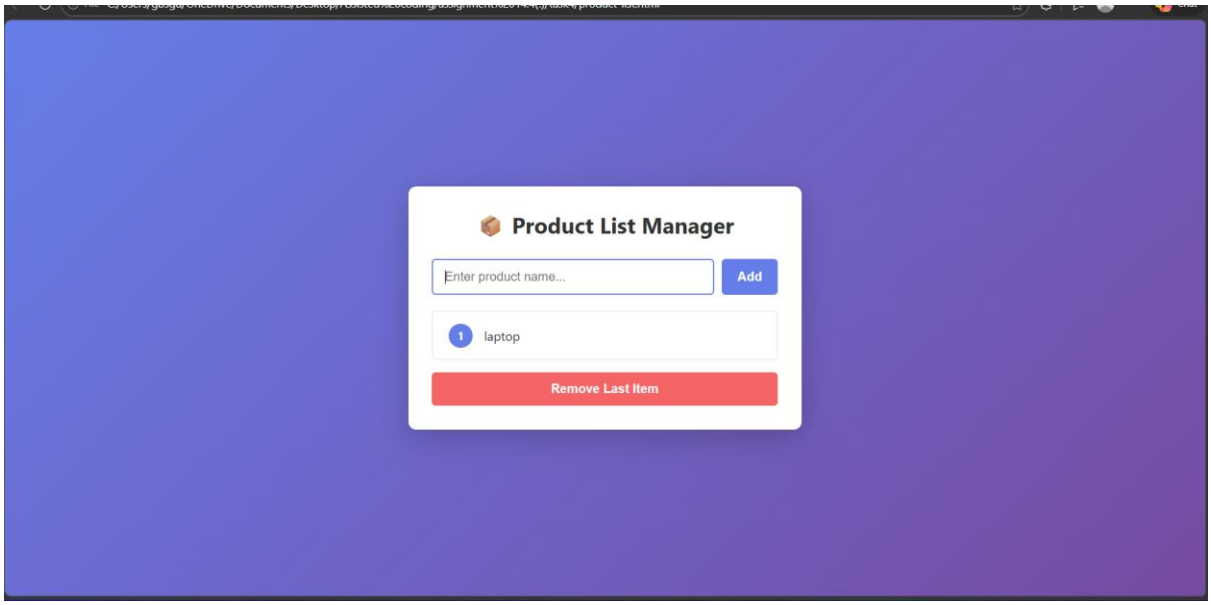
Product Page

Scenario

A product listing page needs to add or remove items dynamically

without refreshing the page.





Task 5: Modal Popup for User Notifications

Scenario

You are building a user notification popup for announcements or alerts

on a website.

notification-modal.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Notification Modal Popup</title>
    <link rel="stylesheet" href="notification-modal.css">
</head>
<body>
    <!-- Button to trigger the notification modal -->
    <button id="showNotificationBtn" class="notification-btn">Show Notification</button>

    <!-- Notification Modal Overlay -->
    <div id="notificationModal" class="modal-overlay">
        <div class="modal-content">
            <!-- Close button -->
            <span class="close-button">&times;</span>

            <!-- Notification Icon (optional) -->
            <div class="notification-icon">
                <svg width="48" height="48" viewBox="0 0 24 24" fill="none" xmlns="http://www.w3.org/2000/svg">
                    <path d="M12 2C10.896 2 10 2.896 10 4V5.586L7.707 8.293C7.317 8.683 7.317 9.317 8 9.707L10 11.414V16C10 17.104 10.896 18 12 18S14 17.104 14 16V11.414L16 9.707C16.683 9.317 16.
                    <path d="M12 20C11.448 20 11 20.448 11 21C11 21.552 11.448 22 12 22S13 21.552 13 21C13 20.448 12.552 20 12 20Z" fill="#4CAF50"/>
                </svg>
            </div>

            <!-- Notification Message -->
            <h2 class="notification-title">Notification</h2>
            <p class="notification-message">
                This is a sample notification message. You can customize this content to display important updates, alerts, or information to your users.
            </p>

            <!-- Action Button (optional) -->
            <button class="action-btn">Got it!</button>
        </div>
    </div>

    <script src="notification-modal.js"></script>
</body>
</html>
```

MODAL POPUP WITH OVERLAY CODE GENERATION

Generate HTML, CSS and JavaScript for a modal popup with a semi-transparent overlay that opens when clicking an Open Modal button and closes when clicking a close button or the overlay with clean and commented code.

Created index.html and styles.css

Opened browser page at file:///c:/Users/gbogu/OneDrive/Documents/Desktop/Assisted%20coding/assignment%2014.4()/task5/index.html

Created script.js

I've created a complete modal popup implementation with HTML, CSS, and JavaScript. Here's what I generated:

Files created:

- index.html - The main HTML structure with the modal
- styles.css - Styling for the modal with semi-transparent overlay
- script.js - JavaScript functionality for opening/closing the modal

Key features:

- Semi-transparent overlay that covers the entire screen
- Modal opens when clicking the "Open Modal" button
- Closes when clicking the "×" close button or clicking outside the modal content
- Clean, responsive design with rounded corners and shadow
- Well-commented code for easy understanding

The modal is now ready to use. I also opened the HTML file in your browser so you can test it immediately. The implementation uses vanilla JavaScript and CSS, with no external dependencies required.

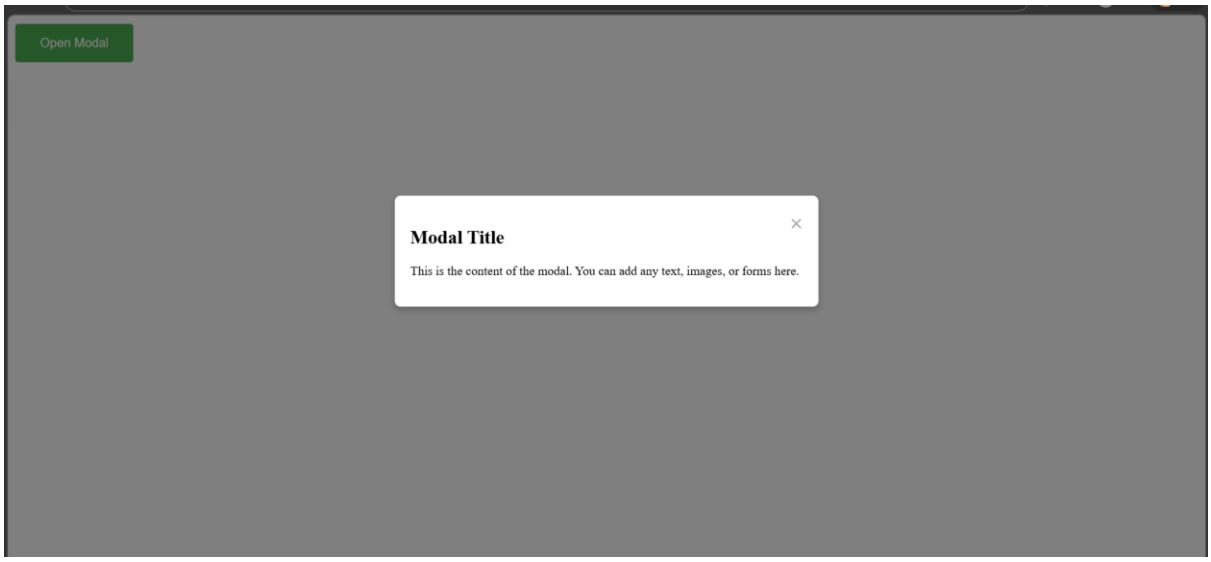
Generate HTML, CSS, and JavaScript code to create a modal popup for website notifications. The modal should include a

index.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Modal Popup Example</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <!-- Button to open the modal -->
    <button id="openModalBtn">Open Modal</button>

    <!-- Modal overlay and content -->
    <div id="modal" class="modal">
        <div class="modal-content">
            <span class="close-btn">&times;</span>
            <h2>Modal Title</h2>
            <p>This is the content of the modal. You can add any text, images, or forms here.</p>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

styles.css > ...

```css
/* Modal Styles */

/* Modal overlay - covers the entire screen with semi-transparent background */
.modal {
    display: none; /* Hidden by default */
    position: fixed; /* Stay in place */
    z-index: 1; /* Sit on top */
    left: 0;
    top: 0;
    width: 100%; /* Full width */
    height: 100%; /* Full height */
    overflow: auto; /* Enable scroll if needed */
    background-color: rgba(0, 0, 0, 0.5); /* Semi-transparent black background */
}

/* Modal content - the actual popup box */
.modal-content {
    background-color: #fefefe; /* White background */
    margin: 15% auto; /* 15% from the top and centered */
    padding: 20px;
    border: 1px solid #888;
    width: 80%; /* Could be more or less, depending on screen size */
    max-width: 500px; /* Maximum width */
    border-radius: 8px; /* Rounded corners */
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2); /* Shadow for depth */
}

/* Close button - positioned at the top right of the modal */
.close-btn {
    color: #aaa;
    float: right;
    font-size: 28px;
    font-weight: bold;
    cursor: pointer;
}

.close-btn:hover,
.close-btn:focus {
    color: black;
    text-decoration: none;
}

/* Button to open modal - basic styling */
#openModalBtn {
    background-color: #4CAF50; /* Green background */
    border: none;
    color: white;
    padding: 15px 32px;
    text-align: center;
    text-decoration: none;
    display: inline-block;
    font-size: 16px;
    margin: 4px 2px;
    cursor: pointer;
    border-radius: 4px;
}

#openModalBtn:hover {
    background-color: #45a049; /* Darker green on hover */
}
```

JS script.js > ...

```javascript
// Get modal element
const modal = document.getElementById('modal');

// Get open modal button
const openModalBtn = document.getElementById('openModalBtn');

// Get close button
const closeBtn = document.getElementsByClassName('close-btn')[0];

// When the user clicks the button, open the modal
openModalBtn.onclick = function() {
    modal.style.display = 'block';
}

// When the user clicks on the close button (x), close the modal
closeBtn.onclick = function() {
    modal.style.display = 'none';
}

// When the user clicks anywhere outside of the modal content, close it
window.onclick = function(event) {
    if (event.target == modal) {
        modal.style.display = 'none';
    }
}
```